

Cloud Computing
Semester 2026-III
Delivery No. 4 — Prototype, Resilience & Final Architecture

Eng. Carlos Andrés Sierra, M.Sc.

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the fourth and final delivery of the *Cloud Computing* course project!

In the three previous deliveries, your team defined what the system must do (Delivery 1), designed a cloud-independent software architecture (Delivery 2), and translated that architecture into a concrete, cost-justified cloud infrastructure plan (Delivery 3). Now it is time to answer the final question: **Does our design actually hold?**

This delivery has three interconnected goals: (1) implement a minimum vertical slice prototype using **AWS Academy** to validate the technical feasibility of your cloud architecture, (2) analyze how the architecture behaves under realistic failure scenarios, and (3) produce a final, revised architecture document that incorporates everything learned during implementation.

General Delivery Definition: Prototype validation and resilience analysis involve demonstrating that a proposed cloud architecture is technically implementable and that its failure behavior is understood and manageable. In this delivery, teams implement a limited but end-to-end working slice of their architecture using **AWS Academy**, analyze three failure scenarios in depth, and revise their complete architecture based on the lessons learned during implementation. The result is a final integrated project that demonstrates both theoretical depth and practical validation.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavir-guezs@udistrital.edu.co*.

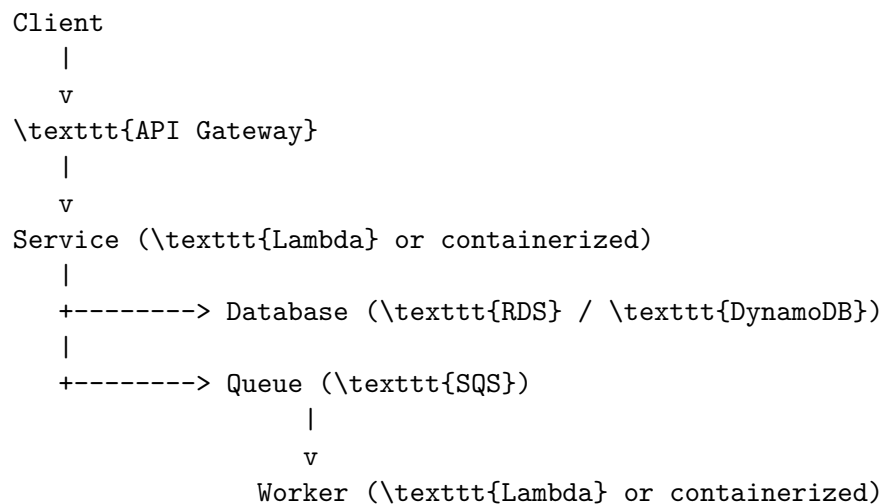
Delivery Scope and Objectives

- **AWS Academy Prototype:** Implement a minimum vertical slice of the architecture using AWS Academy that demonstrates end-to-end technical feasibility.
- **Resilience Analysis:** Analyze three failure scenarios in depth, explaining detection, failover, recovery, data loss, and user impact for each.
- **Final Architecture Revision:** Revise the software and cloud architecture documents from Deliveries 2 and 3 based on implementation learnings.
- **Final Analysis:** Produce a quantitative analysis of the architecture under normal load, peak load, 20× growth, and all three failure scenarios.
- **Final Cost Breakdown:** Update the cost estimate from Delivery 3 with any architectural changes introduced in this delivery.

Required Artifacts and Methodology

1. AWS Academy Prototype — Minimum Vertical Slice:

The prototype must implement a minimum but complete vertical slice of your architecture using AWS Academy. You are **not** required to implement the entire application. The required slice is:



The vertical slice must:

- Be centered on the **most critical or most complex workflow** of your assigned application (e.g., ride matching for Uber, order placement for DoorDash, playback session initiation for Netflix)

- Use **AWS Academy** Learner Lab to provision real AWS resources for the prototype
- Implement the vertical slice following the reference architecture you designed in Delivery 3, using only the services available in your AWS Academy learner account
- Include a setup script (`setup.sh` or equivalent) and **Terraform** or **CloudFormation** that provisions all required AWS resources (queues, tables, Lambda functions, API Gateway routes)
- Demonstrate the complete flow: a client request flows through **API Gateway**, triggers the service, the service writes to the database and publishes to the queue, and the worker consumes the message
- Be reproducible from the repository following only the `README.md` instructions and a valid AWS Academy learner account

Required AWS services to use (select those applicable to your architecture):

- **Amazon API Gateway** — entry point for client requests
- **AWS Lambda** or containerized service — service business logic
- **Amazon DynamoDB** or **Amazon RDS** — data persistence
- **Amazon SQS** — asynchronous queue between service and worker
- **Amazon S3** (optional) — object storage if relevant to the workflow
- **Amazon SNS** (optional) — notification fan-out if relevant to the workflow

2. Resilience Analysis — Three Failure Scenarios:

For each of the following three failure scenarios, provide a detailed analysis covering all five dimensions:

- **Scenario A — One Application Instance Fails:**

One container or Lambda instance of a critical service becomes unresponsive.

- *Detection:* How does the system detect this failure? (ALB health check, ECS task health, CloudWatch alarm?) What is the detection latency?
- *Failover:* What happens automatically? Does the load balancer stop routing to the failed instance? How quickly?
- *Recovery:* How is a replacement instance launched? What triggers the recovery? How long does recovery take?
- *Data Loss:* Is any in-flight request data lost? Are any queued messages affected?
- *User Impact:* What does the end user experience during the failure window? Are any requests failed or retried?

- **Scenario B — One Availability Zone Becomes Unavailable:**

An entire AWS Availability Zone goes offline, taking all resources deployed in that AZ with it.

- *Detection:* How does the system detect AZ-level failure? What is the detection and failover trigger?

- *Failover*: Which components automatically shift to surviving AZs? Which require manual intervention? What is the failover timeline?
- *Recovery*: When the AZ recovers, how does the system rebalance? Are there any split-brain risks during recovery?
- *Data Loss*: For each data store, what is the potential data loss window (RPO) during an AZ failure?
- *User Impact*: What is the estimated downtime or degraded performance window? Which user-facing features are affected?
- **Scenario C — The Database Becomes Temporarily Unavailable:**
The primary database instance becomes unreachable for a period of 2–5 minutes (e.g., due to a failover event, maintenance window, or network partition).
 - *Detection*: How do dependent services detect that the database is unavailable? Connection timeout? Health check failure?
 - *Failover*: If Multi-AZ RDS is used, how long does the automatic failover to the standby replica take? What happens to in-flight queries?
 - *Recovery*: How do services reconnect after the database recovers? Is connection pool draining and reconnection handled automatically?
 - *Data Loss*: What is the RPO for the database under this scenario? Are any committed transactions lost?
 - *User Impact*: Which user-facing operations fail during the database outage? Are write operations queued, rejected, or silently lost? What is the user-visible error?

For each scenario, also define the **architecture changes** (if any) that would reduce the impact of the failure, and justify whether implementing those changes is cost-effective given the availability SLA from Delivery 1.

3. Final Architecture Revision:

Based on the lessons learned during prototype implementation and the resilience analysis, revise the architecture documents from Deliveries 2 and 3. The final architecture package must include:

- **Revised Software Architecture:**
 - Updated service definitions reflecting any changes discovered during implementation
 - Updated data architecture if implementation revealed consistency or schema issues
 - Updated communication patterns if the prototype revealed synchronous/asynchronous trade-off issues
 - Updated or new ADRs documenting decisions made during implementation
- **Revised Cloud Architecture:**
 - Updated cloud architecture diagram incorporating resilience improvements identified in the failure analysis

- Updated compute sizing if the prototype revealed under- or over-provisioning
- Updated networking design if AZ failure analysis revealed gaps
- Updated data infrastructure if the database failure analysis revealed RTO/RPO gaps

4. Final Quantitative Analysis:

Produce a consolidated analysis table covering all operational scenarios:

Scenario	Req/hour	Instance Count	Est. p95 Latency
Normal load			
Peak load			
20× growth			
Instance failure (A)			
AZ failure (B)			
DB failure (C)			

For each failure scenario row, document the degraded capacity available during the failure window and the estimated user impact.

5. Final Cost Breakdown:

Update the cost estimate from Delivery 3 to reflect:

- Any architectural changes introduced in this delivery based on resilience findings
- Final monthly cost at initial workload
- Final monthly cost at 20× growth workload
- Cost delta introduced by resilience improvements (e.g., Multi-AZ RDS vs. single-AZ)

Deliverables and Documentation

1. Final Integrated Project Report (PDF):

- *Executive Summary*: Complete project overview from requirements to validated prototype
- *Prototype Description*: Architecture of the implemented vertical slice, AWS services used, and implementation decisions
- *Prototype Evidence*: Screenshots or terminal output demonstrating the full request flow end-to-end
- *Resilience Analysis*: Full analysis of all three failure scenarios (detection, failover, recovery, data loss, user impact)

- *Revised Software Architecture:* Updated service definitions, data architecture, communication patterns, and ADRs
- *Revised Cloud Architecture:* Updated infrastructure diagram and component configurations
- *Final Quantitative Analysis:* Consolidated scenario analysis table
- *Final Cost Breakdown:* Updated monthly cost estimates for initial and 20× workload
- *Lessons Learned:* What did the implementation reveal that was not visible in the design? What would you design differently?
- *Conclusions:* Does the proposed architecture hold under the specified workload and failure conditions?

2. Prototype Implementation:

- `prototype/infrastructure/` — Terraform or CloudFormation templates to provision AWS resources
- `prototype/setup.sh` (or `setup.py`) — provisions AWS resources in the learner account
- `prototype/src/` — service and worker source code
- `prototype/client/` — test client that demonstrates the complete vertical slice flow
- `prototype/README.md` — step-by-step setup and execution instructions for the AWS Academy environment

3. Final Repository State:

- All delivery folders (`delivery_1/` through `delivery_4/`) complete and organized
- `prototype/` folder with all implementation files
- Root `README.md` updated with the complete project summary and all instructions
- Repository tagged with `v1.0-final` at the final delivery commit

Submission Requirements

- The PDF report file must follow the naming convention: `groupN_delivery4.pdf`, where N is the team number (e.g., `group1_delivery4.pdf`, `group3_delivery4.pdf`)
- Submit the complete Final Integrated Project Report as a single PDF through the designated course platform
- Include the `GitHub` repository link with the final tagged commit (`v1.0-final`)
- All documentation must be in **English** and follow professional technical writing standards
- The prototype must be fully reproducible from the repository following only the `README.md` instructions
- Each team member must be prepared to explain and defend any component of the project during the final presentation

Important Guidelines and Considerations

- **Vertical Slice, Not Full Implementation:** You are not expected to implement the entire application. The vertical slice must be *representative* of your architecture, not *complete*. One well-implemented end-to-end flow is more valuable than a partial implementation of many flows.
- **AWS Academy Constraints:** AWS Academy provides limited, temporary access to AWS services. Plan your prototype carefully within those constraints. Your prototype must be *deployable and executable* within the service limits and runtime of your learner lab.
- **Resilience Analysis Must Be Specific:** “The system will fail over automatically” is not an acceptable resilience analysis. Specify the exact detection mechanism, the time to detection, the time to failover, the recovery trigger, and the user-visible impact. Every claim must be supported by your architecture design.
- **Revisions Are Expected and Valued:** The final architecture revision is not an admission of failure in Deliveries 2 and 3. It is evidence of engineering maturity — the ability to learn from implementation and improve a design. Changes backed on prototype evidence and practical AWS experience are rewarded.
- **Individual Understanding is Evaluated:** During the final presentation, each team member will be questioned individually about any aspect of the project. Ensure that every member understands the entire architecture, not just their individual contribution.

- **Cost of Resilience:** Every resilience improvement has a cost. Multi-AZ adds $2\times$ the instance cost. Read replicas add storage and compute cost. The final cost breakdown must reflect all resilience improvements introduced in this delivery and justify them against the availability SLA from Delivery 1.

Good luck on this final delivery — and take pride in what you have built. You have traveled the complete arc of a cloud architect: from business requirements to software design, to cloud infrastructure, to a working prototype, to a resilience-validated final architecture. That is the professional journey. Well done!